

Nova-16: A Complete 16-Bit Computer System

Architecture, Instruction Set, Language, and Toolchain Specification
- Debashis Baral

1. System Overview

Nova-16 is a complete 16-bit computing system that spans the entire computing stack—from elementary logic primitives to a high-level programming language and operating system libraries. The system is designed to demonstrate how software abstractions emerge systematically from hardware while remaining simple, deterministic, and conceptually clear. At its core, Nova-16 defines a cohesive execution model consisting of a central processing unit (CPU), a well-specified instruction set architecture (Nova ISA), a symbolic assembly language, a stack-based virtual machine layer, a high-level programming language (Luma), and a minimal operating system providing essential runtime services. Each layer is precisely specified and designed to integrate seamlessly with the layers above and below it.

2. Key Features

- Elementary hardware primitives, including logic gates, multiplexers, adders, and sequential storage elements
- Clocked registers and register-based storage, enabling deterministic stateful computation
- Arithmetic Logic Unit (ALU) supporting arithmetic, logical, and comparison operations
- Central Processing Unit (CPU) implementing a fetch–decode–execute execution model
- 16-bit word-oriented architecture with a single, unified address space
- Nova Instruction Set Architecture (ISA) defining binary instruction formats and execution semantics
- Symbolic assembly language providing a human-readable representation of Nova ISA instructions
- Stack-based virtual machine (VM) serving as an intermediate abstraction between high-level language and machine code
- Luma high-level programming language, designed specifically for the Nova-16 execution model
- End-to-end compilation toolchain, including assembler, compiler, and VM translator
- Minimal operating system libraries providing memory management, input/output, and utility services

3. Inspiration and Acknowledgment

The conceptual structure of the Nova-16 system is inspired by the pedagogical framework presented in *The Elements of Computing Systems* by Noam Nisan and Shimon Schocken, which outlines a first-principles approach to constructing complete computer systems.

Nova-16 is an independent implementation that follows this conceptual progression while defining its own system specification, naming, language definitions, toolchain implementation, and design tradeoffs. All components of the Nova-16 system are implemented and documented as a cohesive and standalone computing platform.

Part I — Combinational Logic and Arithmetic Logic Unit (ALU)

Combinational logic forms the stateless computational foundation of the Nova-16 system, used exclusively for data transformation. The Arithmetic Logic Unit (ALU) is the most significant combinational component, defining the core computational capability for arithmetic, logical, and comparison operations.

Design Role and Interface

The ALU is a pure combinational unit:

- It performs computation without memory or side effects.
- It produces deterministic outputs.
- It exposes status information for control-flow instructions.

Its interface includes:

Inputs

- Operand X (16-bit)
- Operand Y (16-bit)
- Control signals selecting the operation to perform

Outputs

- Result (16-bit)
- Status flags:
 - Zero (Z): asserted if the result equals zero
 - Negative (N): asserted if the most significant bit of the result is set

The ALU supports a minimal set of operations like integer addition/subtraction, bitwise logic (AND, OR, NOT), passthrough, and comparison through flag evaluation.

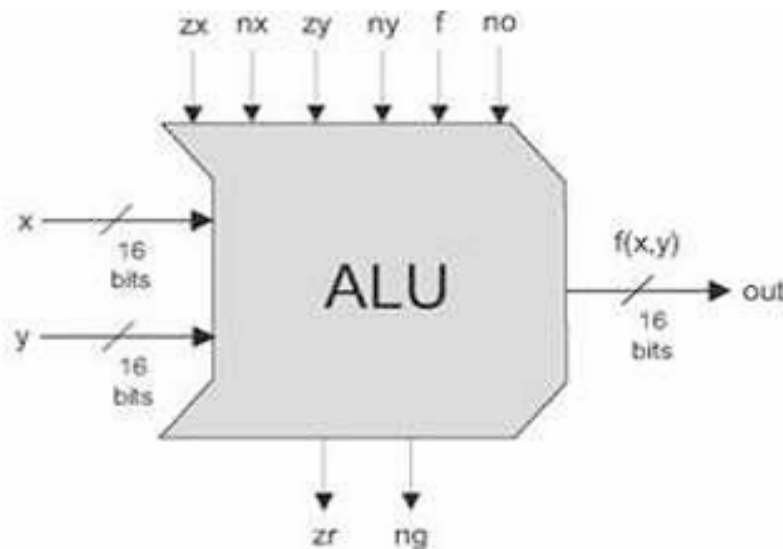


Figure 1: Nova-16 ALU Block Diagram

These bits instruct how to preset the x input		These bits instruct how to preset the y input		This bit selects between + / And	This bit inst. how to postset out	Resulting ALU output
zx	nx	zy	ny	f	no	out=
if zx then x=0	if nx then x=!x	if zy then y=0	if ny then y=!y	if f then out=x+y else out=x&y	if no then out=!out	f(x,y)=
1	0	1	0	1	0	0
1	1	1	1	1	1	1
1	1	1	0	1	0	-1
0	0	1	1	0	0	x
1	1	0	0	0	0	y
0	0	1	1	0	1	!x
1	1	0	0	0	1	!y
0	0	1	1	1	1	-x
1	1	0	0	1	1	-y
0	1	1	1	1	1	x+1
1	1	0	1	1	1	y+1
0	0	1	1	1	0	x-1
1	1	0	0	1	0	y-1
0	0	0	0	1	0	x+y
0	1	0	0	1	1	x-y
0	0	0	1	1	1	y-x
0	0	0	0	0	0	x&y
0	1	0	1	0	1	x y

Figure 2: ALU Control Signal Mapping/Truth Table

This table defines the precise mappings of the control signals that select the specific ALU operation (such as X+Y, X AND Y, 0, 1, etc.) to produce the 16-bit output Result, based on the input operands X and Y.

Control Abstraction and Tradeoffs

Instruction decoding translates Nova ISA instructions into ALU control signals, keeping the ALU hardware independent of instruction encoding. Nova-16 limits ALU complexity, delegating higher-level operations to software libraries for hardware simplicity and predictability. The ALU is designed as stateless, minimal, and decoupled from instruction encoding.

Part II — Sequential Logic and State

Motivation for State

Any system capable of executing programs must preserve intermediate results, track execution progress, and respond predictably to ordered events. This requirement introduces the notion of state. In Nova-16, all stateful behavior is explicitly implemented using sequential logic, ensuring that time-dependent behavior is controlled, observable, and deterministic.

Clocked Versus Unclocked Logic

Sequential logic in Nova-16 is strictly clocked. All state transitions occur only at well-defined clock boundaries. This design choice provides the following guarantees:

- State changes are synchronized across the entire system
- Combinational propagation delays do not affect architectural correctness
- The system can be reasoned about using discrete execution steps

Unclocked or asynchronous storage is intentionally avoided, as it introduces timing ambiguity and complicates both verification and hardware realization.

Time and State Model

Nova-16 adopts a global synchronous clock model:

- Combinational logic continuously computes outputs based on current inputs
- Sequential elements sample and update state only on clock edges
- Each clock cycle represents a single, atomic advancement of system state

D Flip-Flop as the Fundamental Storage Element

The fundamental unit of storage in Nova-16 is the D (Data) Flip-Flop. A D flip-flop captures a single-bit input value on a clock edge and retains that value until the next active clock edge. The D flip-flop is chosen because it provides:

- Explicit separation between data and timing
- Predictable update behavior
- Clean composability into larger storage structures

All higher-level storage components in Nova-16 are constructed from this primitive abstraction.

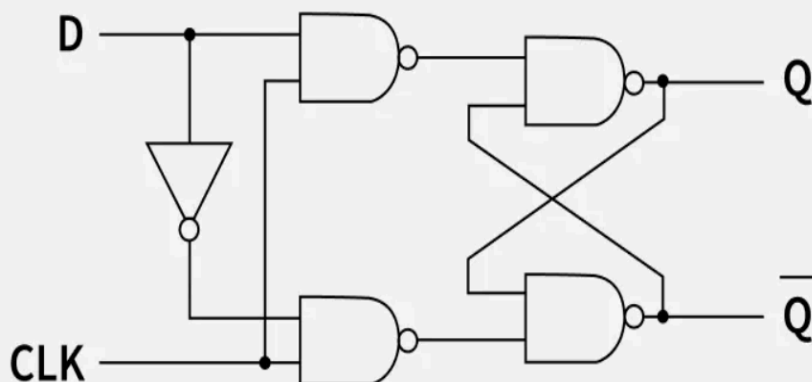


Figure 3: D Flip-Flop

The D flip-flop captures the input value on the active clock edge and holds it stable until the next clock event, forming the atomic unit of state in the Nova-16 system.

Single-Bit Storage

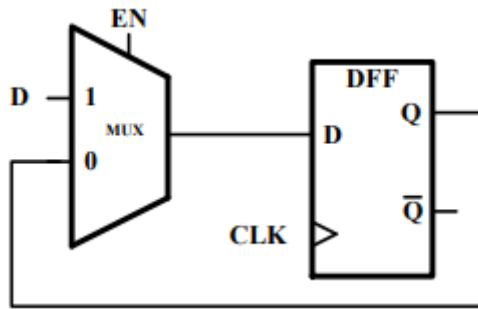


Figure 4: Single Bit Storage

A single D flip-flop constitutes a 1-bit storage cell. Single-bit storage is used internally for:

- Control flags
- Enable signals
- Intermediate state elements
-

Multi-Bit Registers

An N-bit register is formed by grouping multiple D flip-flops and clocking them synchronously. Each flip-flop stores one bit of the register value, and all bits are updated simultaneously.

Registers serve as the primary mechanism for storing:

- Intermediate computation results
- Addresses and counters
- Instruction-related state

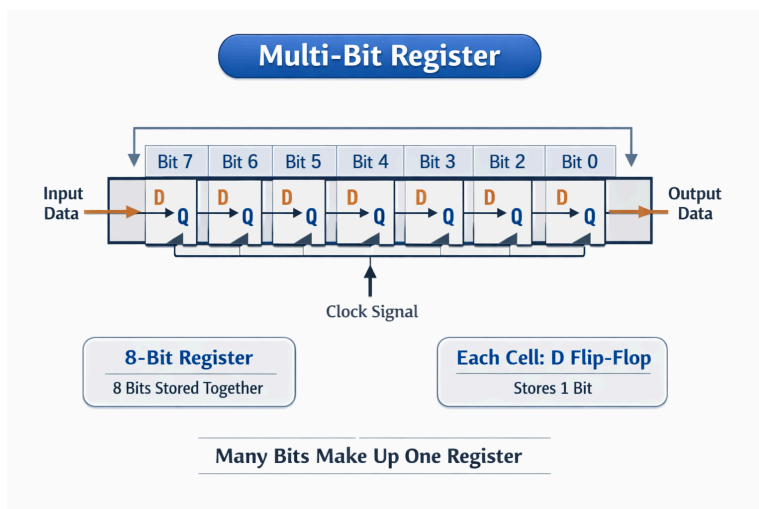


Figure 5: A multi-bit register

By defining registers as width-parameterized structures, Nova-16 maintains architectural consistency across all storage components.

Architectural Storage Registers

Nova-16 defines two primary architectural storage registers, **A** and **D**, each with a fixed width of **16 bits**. These registers serve as the fundamental programmer-visible storage elements and are directly accessible through the Nova ISA.

The **A register** is used for addressing and immediate value storage, while the **D register** functions as a general-purpose data register for arithmetic, logical, and intermediate computation. Both registers are implemented as clocked storage elements with explicit load control and deterministic update semantics.

Part III — Memory Structures and Program Counter

Role of Memory in Nova-16

Registers provide fast, limited storage, but a complete computing system requires a larger, addressable memory to store programs, data, and intermediate results. In Nova-16, this role is fulfilled by Random Access Memory (RAM), which enables address-based storage and retrieval of 16-bit values. RAM in Nova-16 is designed as a structured extension of register storage, preserving the same clocked update semantics while scaling capacity through addressing and selection logic.

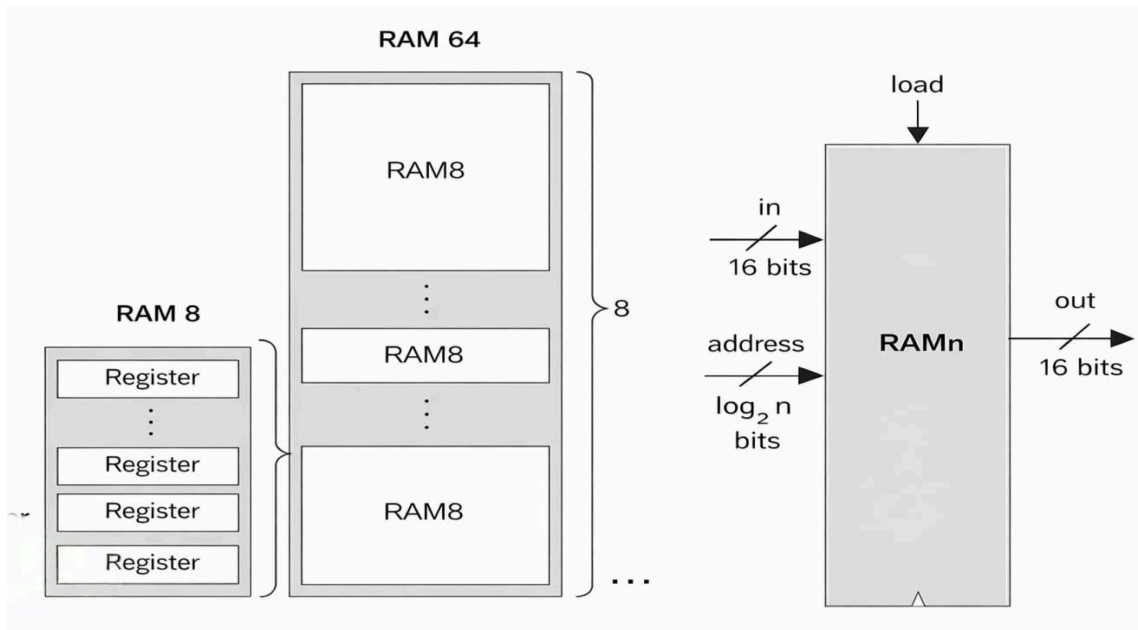


Figure 5: RAM Hierarchy

Scaling Storage: From Registers to RAM

Conceptually, RAM is constructed by organizing multiple registers into an indexed collection, where each register represents a memory word. An address uniquely selects one storage location from this collection. Each memory word:

- Stores 16 bits
- Is independently addressable
- Updates state only under controlled conditions

Address decoding logic ensures that, for any given address, exactly one memory word is selected for write operations.

RAM Behavioral Semantics

- Memory is accessed using a **16-bit address**
- The value at the addressed location is presented on the output
- Writes update memory only at clock boundaries
- Non-selected memory locations remain unaffected

These semantics ensure that memory behavior is independent of physical timing details and consistent across implementations.

Program Counter (PC): Driving Execution Flow

The Program Counter (PC) is a dedicated register that holds the address of the next instruction to be executed. It defines the temporal progression of program execution and serves as the primary control-flow state element in Nova-16.

16-bit Program Counter

Inputs: in[16]

Outputs: inc, load, reset

Function:

If reset = 1 then out = 0

Else if load = 1 then out = in

Else if inc = 1 then out + 1

Else out out

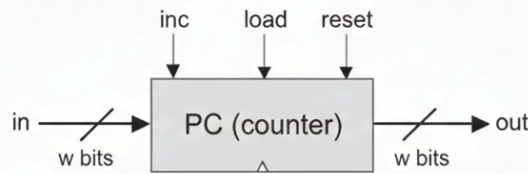


Figure 6 : Program Counter

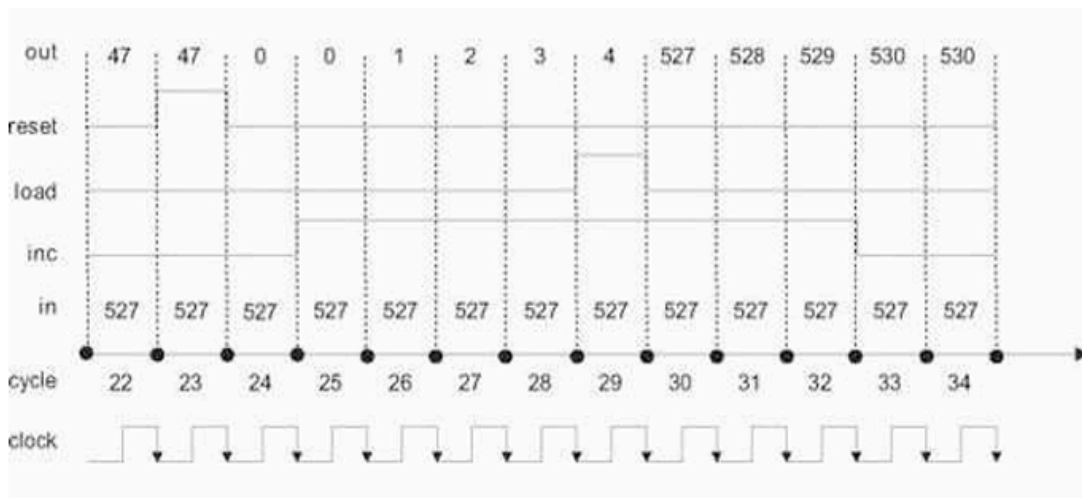


Figure 6 : Program Counter Timing Diagram

Program Counter (PC) — Architectural Design Specification

- The Program Counter is a 16-bit clocked register that holds the address of the next instruction to be executed.
- PC update logic is implemented using chained multiplexers(**mux chaining**) to enforce a clear and deterministic priority among competing update sources.
- An internal incremter continuously computes $PC + 1$, enabling automatic sequential instruction execution without software intervention.
- A load path allows the PC to be explicitly updated with an external address, supporting jump and control-flow instructions.
- A reset path overrides all other update sources and forces the PC to a known initial state (0), ensuring deterministic program startup.
- Update priority is enforced as follows:
 1. Reset (highest priority)
 2. Explicit load (jump target)
 3. Increment (sequential execution)
 4. Hold current value (implicit default)
- All selection decisions are resolved through combinational logic before the clock edge, and the final PC value is committed only at the clock boundary.
- The PC output is continuously available to the instruction memory subsystem, enabling stable instruction fetch throughout the execution cycle.

Part IV — Nova Instruction Set Architecture and Assembly Language

Nova Instruction Set Architecture (ISA)

Role of the ISA

The Nova Instruction Set Architecture (ISA) defines the formal contract between software and hardware in the Nova-16 system. It specifies the binary instruction formats, execution semantics, and register model that all Nova-16-compliant implementations must follow. The ISA is intentionally minimal and deterministic.

ISA Characteristics

- Fixed instruction width: 16 bits
- Word-oriented architecture: all data values are 16 bits
- Single, unified address space
- Explicit control flow through jump semantics

Register Model

Nova-16 exposes two primary architectural registers:

- **A register (16-bit):** Used as an address register and immediate value holder.
- **D register (16-bit):** Used as a general-purpose data register for computation.

The Program Counter (PC) is an internal control register responsible for instruction sequencing.

Instruction Classes

The Nova ISA defines two instruction formats, selected by the most significant bit:

- **A-type instruction** (address / constant loading)
- **C-type instruction** (computation, storage, and control flow)

Instruction Formats and Semantics

A-Type Instruction

The A-type instruction is used to load a 15-bit value (a constant or an address) directly into the A register. It is non-computational and serves to introduce constant values or to indirectly specify memory addresses and jump targets.

- **Format (16 bits):**
0 v v v v v v v v v v v v v v

C-Type Instruction

The C-type instruction is the primary computational instruction. It specifies **what to compute** (via the ALU), **where to store the result** (A, D, or Memory at address A), and **what instruction to execute next** (conditional jump).

- **Format (16 bits):**
1 1 1 a c1 c2 c3 c4 c5 c6 d1 d2 d3 j1 j2 j3

The instruction's fields tightly couple the ISA to the ALU and register load logic, ensuring a simple and deterministic execution model. Jumps are based on the ALU's status flags (Negative, Zero) and cause the PC to load the value currently stored in the A register.

Nova Assembly Language

Role of Assembly

The Nova assembly language is a symbolic, human-readable representation of the Nova ISA. Each assembly instruction maps directly to one machine instruction. It exists purely for programmer convenience, allowing the use of symbols and labels for constants, addresses, and jump targets. The assembler translates this symbolic code into binary machine code.

Assembly Instruction Types

- **A-instruction:** `@value` (Loads a constant or address into the A register.)
- **C-instruction:** `dest = comp ; jump`

Memory and I/O Access

Memory access is indirect and always occurs through the A register. The symbol `M` refers to the memory word addressed by `A`. Screen and keyboard devices are accessed through memory-mapped I/O.

Assembly Language Examples

Instruction	Type	Description
@100	A-Instruction ▾	Load the constant value 100 into the A register.
@LOOP	A-Instruction ▾	Load the memory address associated with the symbol <code>LOOP</code> into the A register.
D=A	C-Instruction ▾	Compute <code>D=A</code> (Passthrough) and store the result in the D register.
M=D	C-Instruction ▾	Compute <code>D</code> (Passthrough) and store the result in the memory location addressed by A.
D=A-M	C-Instruction ▾	Compute <code>A - Memory[A]</code> and store the result in the D register.
D;JGT	C-Instruction ▾	If the current computation result is greater than zero, jump to the address in the A register.
M=D;JMP	C-Instruction ▾	Store D into Memory[A], and then perform an unconditional jump to the address in the A register.

Architectural Design Rationale

The Nova ISA and assembly language are designed around the principles of **simplicity, explicitness, and determinism**. By maintaining a minimal instruction set and fixed formats, the system reduces hardware cost while ensuring that every operation has a predictable and fully specified effect, making Nova-16 an ideal platform for learning the complete computing stack.

NOVA-16 Instruction Set Architecture (ISA) Reference

Computation (comp) Field

(when a=0)

(when a=1)

comp mnemonic	c1	c2	c3	c4	c5	c6	comp mnemonic
0	1	0	1	0	1	0	
1	1	1	1	1	1	1	
-1	1	1	1	0	1	0	
D	0	0	1	1	0	0	
A	1	1	0	0	0	0	M
!D	0	0	1	1	0	1	
!A	1	1	0	0	0	1	!M
-D	0	0	1	1	1	1	
-A	1	1	0	0	1	1	-M
D+1	0	1	1	1	1	1	
A+1	1	1	0	1	1	1	M+1
D-1	0	0	1	1	1	0	
A-1	1	1	0	0	1	0	M-1
D+A	0	0	0	0	1	0	D+M
D-A	0	1	0	0	1	1	D-M
A-D	0	0	0	1	1	1	M-D
D&A	0	0	0	0	0	0	D&M
D A	0	1	0	1	0	1	D M

Destination Field

d1	d2	d3	Mnemonic	Destination (where to store the computed value)
0	0	0	null	The value is not stored anywhere
0	0	1	M	Memory[A] (memory register addressed by A)
0	1	0	D	D register
0	1	1	MD	Memory[A] and D register
1	0	0	A	A register
1	0	1	AM	A register and Memory[A]
1	1	0	AD	A register and D register
1	1	1	AMD	A register, Memory[A], and D register

Jump Field

j1 (out < 0)	j2 (out = 0)	j3 (out > 0)	Mnemonic	Effect
0	0	0	null	No jump
0	0	1	JGT	If out > 0 jump
0	1	0	JEQ	If out = 0 jump
0	1	1	JGE	If out ≥ 0 jump
1	0	0	JLT	If out < 0 jump
1	0	1	JNE	If out ≠ 0 jump
1	1	0	JLE	If out ≤ 0 jump
1	1	1	JMP	Jump

Part V — Execution Model and CPU Design

Von Neumann Architecture Model

Nova-16 is implemented as a **Von Neumann architecture**, in which instructions and data share a single, unified memory address space and communicate with the CPU over common data and address pathways. The Program Counter supplies instruction addresses, while the same memory subsystem is accessed for data reads and writes during instruction execution. This architectural choice simplifies system design by unifying memory access semantics and minimizing hardware duplication.

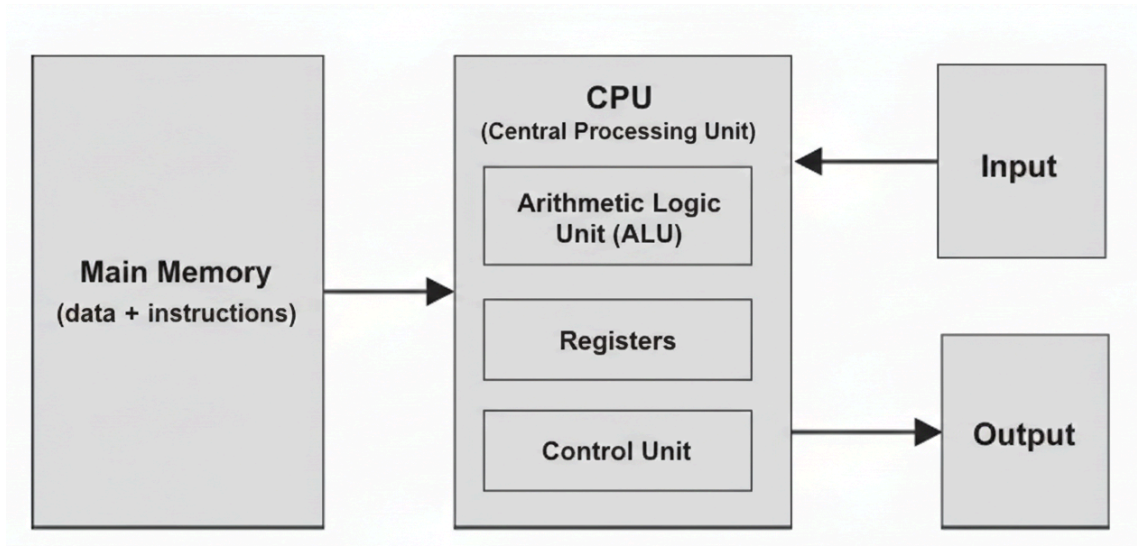


Figure 9: Nova-16 Von Neumann Architecture

A unified memory subsystem stores both instructions and data, accessed by the CPU through shared address and data paths. Control flow is driven by the Program Counter, while memory-mapped I/O devices reside within the same address space.

Architectural Rationale

- A single memory model reduces hardware complexity
- Unified addressing simplifies the instruction set and control logic
- Memory-mapped I/O integrates peripherals without special instructions
- Deterministic access sequencing avoids ambiguity in execution behavior

This model aligns with Nova-16's goals of clarity, minimalism, and implementation transparency, while remaining sufficient to support structured programs and operating system abstractions.

Execution Model

Instruction Lifecycle

Nova-16 follows a single-cycle, synchronous execution model. Each instruction completes its entire effect within one clock cycle. The execution model is deterministic and globally synchronized by the system clock. The instruction lifecycle consists of the following conceptual phases:

- **Fetch**
 - The Program Counter (PC) provides the address of the next instruction
 - Instruction memory outputs the instruction word
 - Instruction value remains stable for the duration of the cycle

- **Decode**
 - Instruction bits are decoded combinatorially
 - Instruction type (A or C) is determined
 - Control signals are derived directly from instruction fields
- **Execute**
 - ALU performs the specified computation
 - Operands are sourced from registers or memory
 - Status flags (Z, N) are generated
- **Write-back**
 - Destination registers or memory are updated at the clock edge
 - Program Counter is updated based on control-flow outcome

All phases overlap within the same clock cycle, enabled by the absence of intermediate pipeline registers.

Fetch-Execute Cycle

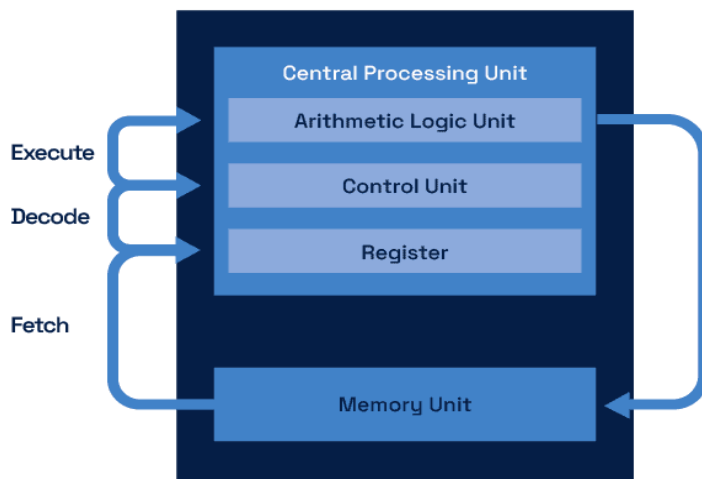


Figure 10: Fetch-Execute Cycle Diagram

Control Flow Semantics

Nova-16 supports two execution modes:

- **Sequential execution**
 - PC automatically increments to PC + 1
 - Occurs when no jump condition is satisfied
- **Branching and jumps**
 - Jump condition is evaluated using ALU status flags
 - If satisfied, PC is loaded with the value in the A register

Jump target addresses are never encoded directly in C-instructions; instead, control-flow targets are always supplied indirectly through the A register.

Timing Assumptions

- All architectural state updates occur only at clock edges
- Combinational logic stabilizes within one clock cycle
- Instruction memory output is stable for the entire cycle
- No instruction spans multiple cycles

This model ensures simple reasoning and straightforward hardware realization.

All datapath paths are combinational, with registers acting as the only state-holding elements. The CPU datapath connects registers, ALU, memory interface, and control logic, enabling single-cycle instruction execution.

Control Unit

Nova-16 uses hardwired control logic derived directly from instruction bits. Control logic responsibilities:

- Instruction classification (A vs C)
- ALU control signal generation
- Register load enable generation
- Memory write control
- Jump condition evaluation
- Program Counter update control

All control signals are computed combinatorially from the instruction and ALU status flags.

Control Signal Flow

Control signals coordinate datapath behavior:

- ALU control bits drive computation
- Destination bits drive register and memory writes
- Jump bits combine with ALU flags to control PC load
- Reset signal overrides all control paths

Control signal generation does not rely on sequencing or micro-operations.

Instruction fields and ALU flags combine to generate control signals that drive datapath updates and control flow.

Clock Synchronization

- All registers sample inputs on the clock edge
- No combinational feedback loops exist
- Datapath values stabilize before the clock edge
- State transitions are atomic and deterministic

Clocking acts as the sole mechanism for state progression.

Design Tradeoffs

- **Simplicity vs Performance**
 - Single-cycle execution simplifies design and verification
 - No pipelining or parallelism is implemented
 - Performance is traded for clarity and determinism

These tradeoffs align with Nova-16's goal of architectural clarity and implementation transparency.

Part VI — Complete Computer System Integration

System Overview

This section presents the complete Nova-16 computer system by integrating the CPU, memory subsystem, and memory-mapped I/O devices into a single executable platform. At this level, all previously defined components operate together to form a functional stored-program computer. Nova-16 follows a Von Neumann execution model, where instructions and data reside in a single, unified memory address space and are accessed through a shared memory interface.

Nova-16 System Specifications and Memory Map

Component	Function	Size	Address Range
Instruction Memory (ROM)	Stores program instructions.	32K words (16-bit)	0x0000 – 0x7FFF
Data Memory (RAM)	Stores program data and stack.	16K words (16-bit)	0x0000 – 0x3FFF
Screen (I/O)	Memory-mapped graphical display.	8K words (16-bit)	0x4000 – 0x5FFF
Keyboard (I/O)	Single memory location for input status.	1 word (16-bit)	0x6000

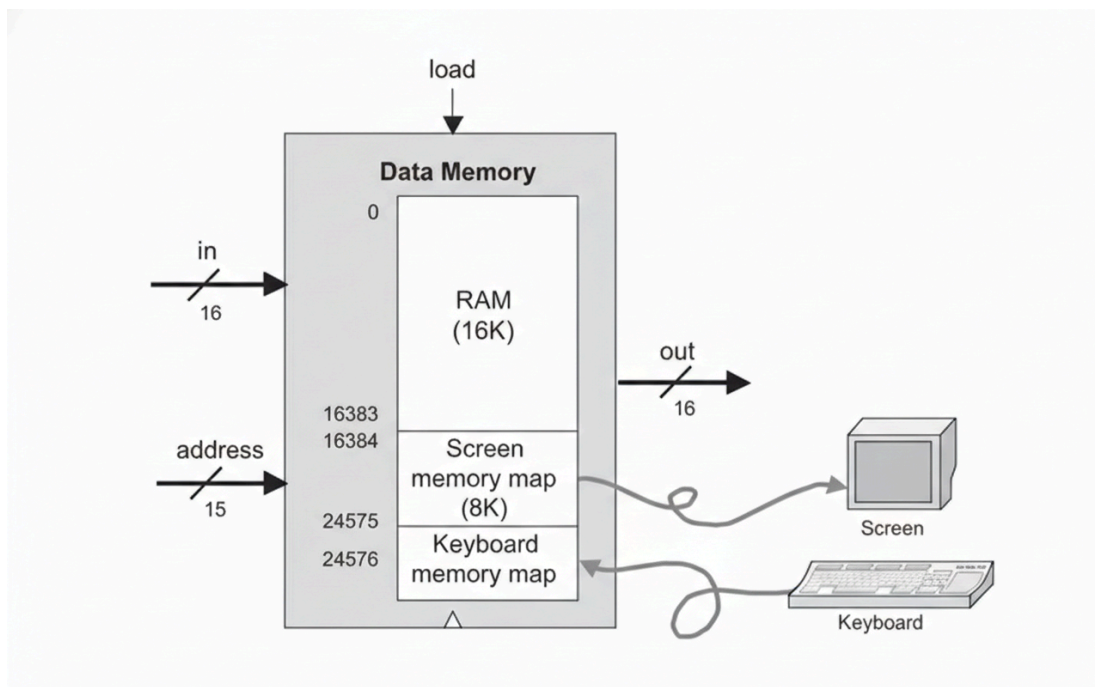


Figure 12: Memory Map and I/O Integration

The unified memory subsystem combines RAM, screen, and keyboard into a single address space. I/O devices are accessed through memory-mapped regions, eliminating the need for special I/O instructions.

External Device Interface

The computer exposes two external devices:

- **Screen:** Written via memory-mapped addresses to update display pixels.
- **Keyboard:** Read via a fixed memory-mapped location representing the most recent key event.

Both devices operate synchronously with the memory subsystem and are accessed using standard memory read and write operations.

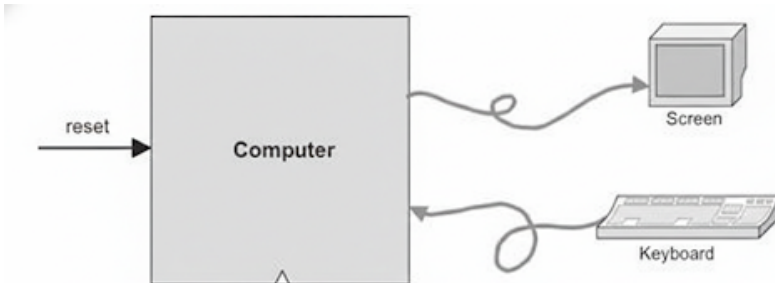


Figure 13: External I/O Connections Caption:

The Nova-16 computer interfaces with external screen and keyboard devices through memory-mapped I/O, enabling interaction using ordinary memory instructions.

CPU–Memory–Instruction Integration

The full system execution loop operates as follows:

1. The Program Counter (PC) supplies the address of the next instruction
2. The Instruction Memory (ROM) outputs the instruction word
3. The CPU decodes and executes the instruction
4. Data memory and I/O are accessed as required
5. The PC updates to the next sequential or jump target address

Instruction fetch, execution, and state update are synchronized by the system clock, ensuring deterministic behavior.

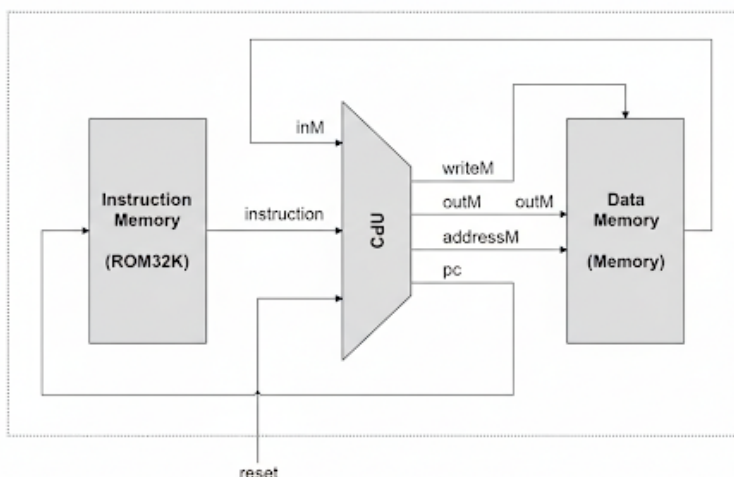


Figure 14: Complete Nova-16 Computer Architecture

The complete Nova-16 system integrates instruction memory, CPU, and data memory into a single execution platform. The CPU mediates all data movement and control flow between memory and external devices.

Part VII — Software Stack Overview

Overview of the Nova Software Ecosystem

Nova-16 is supported by a layered software ecosystem that bridges high-level programming abstractions and low-level hardware execution. This stack is designed to clearly separate concerns, simplify translation between abstraction levels, and enable independent development and verification of each component. Rather than compiling high-level programs directly to machine code, Nova-16 adopts a staged compilation model built around a virtual machine abstraction. This approach reduces compiler complexity and provides a clean, modular software architecture.

Software Layers

The Nova software stack is organized into the following layers, from highest to lowest abstraction:

- 1. Luma — High-Level Programming Language**
Luma is the programmer-facing language used to write structured, object-based programs.
- 2. Nova VM Language — Stack-Based Intermediate Representation**
The Nova VM language serves as an intermediate, platform-independent representation of program behavior. It abstracts away hardware details and expresses computation using a stack-based execution model.
- 3. Nova Assembly — Symbolic Machine Language**
Nova assembly provides a human-readable, symbolic representation of Nova ISA instructions.
- 4. Nova OS Library — System Services**
The operating system library provides essential runtime services such as memory allocation, I/O handling, string manipulation, and basic utilities. These services are implemented at the VM and assembly levels and are exposed to Luma programs through standardized interfaces.
- 5. Nova-16 Hardware — Execution Substrate**
The hardware layer executes binary machine code produced by the assembler, implementing the Nova ISA and providing memory-mapped I/O and deterministic execution semantics.

Compilation and Translation Pipeline

Program translation in Nova-16 proceeds through a multi-stage pipeline:

1. Luma Compiler

Translates Luma source code into Nova VM instructions. This stage handles:

- Syntax and semantic analysis
- Scope and symbol resolution
- High-level control flow translation

2. VM Translator

Converts Nova VM instructions into Nova assembly code. This stage maps stack-based operations to register-based machine instructions while preserving program semantics.

3. Assembler

Translates Nova assembly into binary machine code executable by Nova-16 hardware. Symbol resolution and instruction encoding occur at this stage.

This staged pipeline ensures correctness, modularity, and ease of implementation.

Rationale for the Virtual Machine Layer

The inclusion of a stack-based virtual machine is a deliberate architectural decision. Direct compilation from a high-level language to machine code would require the compiler to manage low-level concerns such as register allocation, instruction scheduling, and hardware-specific constraints. By introducing a VM layer:

- High-level language translation is simplified
- Hardware details are isolated to the VM translator
- The compiler remains platform-independent
- Multiple high-level languages can target the same VM
- Verification and debugging are significantly easier

The VM acts as a stable semantic bridge between language design and hardware execution.

Design Philosophy

The Nova software stack is guided by the following principles:

- **Separation of concerns:** Each layer has a well-defined responsibility and interface.
- **Platform independence:** High-level language tools are decoupled from hardware details.
- **Simplicity over micro-optimization:** Clarity and correctness are prioritized over low-level performance tuning.
- **Deterministic compilation:** Each translation stage produces predictable, fully specified output.

This layered approach enables independent evolution of the language, tools, and hardware while preserving system coherence.

Part VIII — Luma High-Level Language Specification (Summary)

Overview

Luma is a statically typed, object-based high-level programming language specifically designed for the Nova-16 computing system. It acts as the primary programmer-facing layer, enabling structured program development while ensuring a clear and deterministic mapping to the underlying hardware execution model. This section summarizes its architectural role and design principles.

Language Design Goals

Luma is designed to prioritize simplicity, determinism, and compiler-friendliness:

- **Minimal but expressive:** Provides a compact set of constructs for structured programming and abstraction.
- **Compiler-friendly:** Optimized for deterministic parsing and straightforward compilation to a stack-based virtual machine.
- **Object-based abstraction:** Programs are organized using classes, fields, and subroutines for modular design.
- **Deterministic execution semantics:** All behavior is fully specified and independent of implementation details.

Core Language Model

- **Variables:** Statically typed with explicit declarations.
- **Structure:** Class-based program structure with lexically scoped variables.
- **Subroutines:** Categorized as constructors, methods, and functions.
- **Control Flow:** Explicit constructs only (`if`, `while`, `return`).
- **Computation:** Expression-based using a fixed operator set.
- **Constraints:** Luma prohibits implicit conversions or side effects outside explicitly defined constructs.

Execution Semantics (High-Level)

- **Subroutine Calls:** Create isolated execution contexts.
- **Local Variables:** Follow stack-based lifetimes.
- **Method Calls:** Receive an implicit object reference (`this`).
- **Object Construction:** Explicitly allocates memory and initializes fields.
- **Control Flow:** Proceeds sequentially unless altered by conditional or loop constructs.

Formal Specification Reference

This document provides a language architecture summary only. The complete and authoritative specification of Luma—including lexical structure, formal grammar, syntax rules, and language examples—is defined in the:

Luma Language Handbook

(Official Language Reference and Grammar Specification)

https://docs.google.com/document/d/1prphT3w0hAw_48aYSwMlf1kxknqNGyrL7LYtg4_5_eE/edit?usp=sharing

All Luma implementations must conform to the rules defined in the official handbook.

Design Summary

Luma is a deliberately constrained, precisely defined programming language that prioritizes clarity, determinism, and architectural alignment over feature richness. Its design enables reliable compilation, predictable execution, and seamless integration with the Nova-16 computing stack.

Part IX — Luma Virtual Machine (LVM) Language Specification (Summary)

Overview

The Luma Virtual Machine Language (LVM) is a stack-based intermediate representation designed for the Nova-16 software toolchain. It serves as the translation boundary between the Luma high-level language and Nova-16 assembly.

Design Motivation

The LVM layer was introduced to address key challenges in compiler construction and system scalability:

- **Simplified Compilation Pipeline:** Provides a structured intermediate form that simplifies translation from complex high-level constructs to low-level assembly.
- **Portability and Retargetability:** Enables a single Luma compiler to target multiple architectures by only replacing the VM-to-assembly translation stage.
- **Deterministic Execution Model:** Defines precise stack behavior, memory access rules, and control-flow semantics, eliminating ambiguity in translation.

Position in the Software Stack

LVM acts as a semantic bridge, positioned to retain Luma-level abstractions while remaining low enough to map efficiently to simple assembly sequences:

- **Retains High-Level Abstractions:** Subroutine calls, local variables, structured control flow, and expression evaluation.
- **Maps to Low-Level Execution:** Expresses computation through stack manipulation and explicit memory segments, avoiding language features that require complex runtime interpretation.

Core Execution Model

- **Stack-Based Computation:** All arithmetic, logical, and control operations operate on an implicit operand stack.
- **Segmented Memory Model:** Memory is abstracted into logical segments (e.g., local, argument, static, this, that), which are deterministically mapped to physical RAM during translation.
- **Explicit Control Flow:** Program flow is expressed using labels and conditional/unconditional branching commands.
- **Function Call Discipline:** Subroutine calls follow a strict calling convention, ensuring isolated execution contexts and predictable stack frame layout.

Luma Virtual Machine Language Specification

(Official VM Instruction Set & Grammar Reference)

<https://docs.google.com/document/d/1sCWH4G3m9Hlr-MxUKcQPFL48006iVWgfMZPdZRcBRzo/edit?usp=sharing>